LATIHAN TERSTRUKTUR

PERTEMUAN 4: IMPLEMENTASI UNIT TESTING DENGAN GITHUB ACTIONS

Nama : Selpiya

NIM : 3224007

Prodi : Sistem Informasi

Kelas : Reguler Pagi

Matkul : Implementasi Pengujian Perangkat Lunak

Tanggal : 10 Oktober 2026

Bagian A: Git & Githuub Dasar

Soal 1 – Git & Github Dasar

1. **Penjelasan perbedaan antara Centralized VCS dan Distributed VCS. Beserta 2 contohnya**

   Version Control System (VCS) merupakan sistem yang digunakan untuk mencatat dan mengelola perubahan pada file atau sekumpulan file dari waktu ke waktu. Dengan VCS, setiap perubahan pada kode dapat dilacak sehingga developer dapat mengetahui apa yang diubah, kapan perubahan dilakukan, dan dapat kembali ke versi sebelumnya apabila terjadi kesalahan. Dalam pengembangan perangkat lunak, VCS juga mendukung kolaborasi, branching, merging, backup, serta *traceability* terhadap perubahan kode.

   Berdasarkan cara repository dan riwayat versinya disimpan, VCS dapat dibedakan menjadi Centralized Version Control System (CVCS) dan Distributed Version Control System (DVCS).

   a. Centralized Version Control System (CVCS)

      Centralized VCS adalah sistem version control yang menggunakan satu server pusat sebagai tempat penyimpanan utama seluruh versi dan riwayat perubahan repository. Developer mengakses repository tersebut melalui server pusat untuk mengambil maupun mengirimkan perubahan. Dalam materi dosen dijelaskan bahwa pada CVCS, developer bekerja dari server pusat dan sistem ini mempunyai risiko *single point of failure*.

      Artinya, ketergantungan terhadap server pusat relatif tinggi. Jika server pusat mengalami gangguan atau tidak dapat diakses, aktivitas pengelolaan repository juga dapat terganggu. Selain itu, kemampuan bekerja secara offline menjadi lebih terbatas karena data version control utama berada pada server pusat.

      Contoh Centralized VCS:

      1) SVN atau Subversion
      2) CVS atau Consurrent Versions System

   b. Distributed Version Control System (DVCS)

      Distributed VCS adalah sistem version control di mana setiap developer mempunyai salinan lengkap repository beserta riwayat perubahannya pada komputer lokal. Dengan konsep tersebut, pekerjaan version control tidak sepenuhnya bergantung pada satu server pusat.

      Pada DVCS, developer dapat melakukan banyak aktivitas version control secara lokal, sehingga memungkinkan pekerjaan dilakukan dalam kondisi offline. Selain itu, tidak terdapat ketergantungan yang sama terhadap satu server sebagai *single point of failure*,

dan proses branching juga lebih mudah dilakukan. Dalam materi, Git dijelaskan sebagai jenis DVCS yang digunakan secara luas dalam pengembangan perangkat lunak modern.

Contoh Distributed VCS:

1) Git
2) Mercurial

Perbedaan perbandingannya:

Aspek	Centralized VCS	Distributed VCS
Penyimpanan repository	Repository utama disimpan pada satu server pusat	Setiap developer mempunyai salinan lengkap repository
Ketergantungan server	Sangat bergantung pada server pusat	Tidak sepenuhnya bergantung pada server pusat
Bekerja secara offline	Terbatas	Dapat dilakukan
Branching	Relatif lebih bergantung pada sistem pusat	Lebih mudah dilakukan
Contoh	SVN, CVS	Git, Mercurial

Jadi, perbedaan utama keduanya terletak pada arsitektur penyimpanan repository. Centralized VCS berpusat pada satu server, sedangkan Distributed VCS memberikan setiap developer salinan lengkap repository. Model distributed inilah yang digunakan oleh Git.

2. 5 Git commands yang wajib dikuasai beserta fungsinya

Lima perintah yang menurut saya paling mendasar untuk dikuasai adalah *git init, git add, git commit, git push, dan git pull*. Kelima command tersebut mewakili proses mulai dari membuat repository lokal, mempersiapkan perubahan, menyimpan perubahan, sampai melakukan sinkronisasi dengan remote repository.

1) Git init

   Command:

```
git init
```

git init digunakan untuk menginisialisasi atau membuat repository Git baru pada suatu folder lokal. Ketika sebuah folder project belum menggunakan Git, perintah ini digunakan agar folder tersebut mulai dikelola oleh Git. Setelah dijalankan, Git dapat mulai mencatat perubahan terhadap file-file yang terdapat dalam project.

2) Git add

Command:

```
git add .
```

git add digunakan untuk menambahkan file atau perubahan ke staging area sebelum perubahan tersebut disimpan melalui proses commit. Staging area dapat dipahami sebagai tahap persiapan. Developer memilih perubahan mana yang akan dimasukkan ke commit berikutnya. Jika menggunakan tanda titik (.), seluruh perubahan yang relevan pada direktori kerja akan ditambahkan ke staging area.

Apabila hanya ingin menambahkan satu file tertentu, dapat digunakan:

```
git add README.md
```

Jadi, *git add* belum menyimpan perubahan sebagai sebuah versi permanen dalam riwayat Git, tetapi mempersiapkan perubahan untuk proses commit.

3) Git commit

Command:

```
git commit -m "pesan commit"
```

git commit digunakan untuk menyimpan perubahan yang sudah berada pada staging area ke dalam riwayat repository Git. Commit dapat dianggap sebagai catatan atau *snapshot* dari kondisi project pada suatu titik tertentu. Pesan pada commit sebaiknya menjelaskan perubahan yang dilakukan sehingga riwayat project mudah dipahami.

Dengan adanya commit, perubahan dapat dilacak sehingga developer dapat mengetahui perkembangan project dari waktu ke waktu.

4) Git push

Command:

```
git push origin main
```

Pada command tersebut:

- *origin* menunnukan remote repository yang digunakan.
- *main* menunjukkan branch tujuan.

git push digunakan untuk mengirim commit dari repository lokal ke remote repository, misalnya repository yang berada di GitHub. Setelah melakukan perubahan dan commit pada komputer lokal, perubahan tersebut belum otomatis tersedia di GitHub. Oleh karena itu, git push digunakan untuk mengirim commit lokal ke repository remote.

Setelah push berhasil, perubahan yang sebelumnya hanya terdapat pada repository lokal juga akan tersedia pada repository GitHub.

5) Git pull

Command:

```
git pull origin main
```

git pull digunakan untuk mengambil perubahan terbaru dari remote repository dan membawanya ke repository lokal. Command ini penting terutama ketika sebuah project dikerjakan secara kolaboratif. Misalnya anggota tim lain telah melakukan perubahan dan melakukan push ke GitHub, maka developer lain dapat menggunakan git pull agar repository lokalnya mempunyai versi terbaru. Dengan demikian, repository lokal dapat tetap sinkron dengan perubahan yang terdapat pada branch main di remote repository.

3. Fungsi dari .gitignore dalam repository Git

File .gitignore berfungsi untuk menentukan file atau folder tertentu yang tidak perlu dimasukkan atau di-commit ke dalam repository Git. Tidak semua file yang terbentuk dalam sebuah project perlu disimpan di repository. Dalam proses pengembangan perangkat lunak sering terdapat file yang dihasilkan secara otomatis oleh proses build, IDE, sistem operasi, coverage tool, maupun environment tertentu. File-file tersebut tidak selalu berkaitan langsung dengan source code utama dan dapat membuat repository menjadi tidak rapi apabila semuanya ikut disimpan.

Sebagai contoh, pada project Java berbasis Maven dapat dibuat file .gitignore seperti berikut:

```
# Maven
target/

# IDE
.idea/
*.iml
.vscode/

# Operating System
.DS_Store
Thumbs.db

# Coverage
*.exec
```

Penjelasannya adalah:

- target/ merupakan direktori hasil proses build Maven sehingga tidak perlu dimasukkan ke repository.
- .idea/, *.iml, dan .vscode/ merupakan file atau konfigurasi yang berkaitan dengan IDE atau editor.
- .DS_Store dan Thumbs.db merupakan file yang dapat dihasilkan oleh sistem operasi.
- *.exec dapat merupakan file yang berkaitan dengan hasil atau data coverage.

Dengan menggunakan .gitignore, repository dapat lebih bersih, terorganisir, dan fokus pada file yang memang diperlukan dalam pengembangan project. Hal ini juga mendukung struktur project yang lebih baik karena source code, kode pengujian, workflow GitHub Actions, dan konfigurasi penting tetap tersimpan, sedangkan file hasil generate atau konfigurasi lokal yang tidak diperlukan dapat dikecualikan.

Soal 2 – Praktik Git Commands

Pada praktik ini, saya membuat repository GitHub dengan nama **latihan-git-3224007**, kemudian melakukan proses clone repository ke komputer lokal, membuat file README.md, melakukan commit dan push, membuat branch baru, menambahkan file pada branch tersebut, melakukan push branch, kemudian menggabungkan branch feature-test ke branch main.
Urutan praktik ini sesuai dengan materi Pertemuan 4 yang menjelaskan bahwa penggunaan Git dimulai dari proses setup repository, clone ke lokal, commit dan push, kemudian dapat dilanjutkan dengan penggunaan branch dan merge.

1. **Membuat Repository baru di Github**

   Repository dibuat melalui halaman GitHub dengan nama:

   ```
   latihan-git-3224007
   ```

   Repository dibuat terlebih dahulu dalam keadaan kosong agar file README.md dapat dibuat melalui repository lokal sesuai urutan praktik pada soal.

   Pada langkah ini tidak digunakan Git command karena pembuatan repository dilakukan melalui **GitHub Web.**

2. **Clone repository ke komputer loakal**

   - Sebelum melakukan clone, saya menentukan direktori penyimpanan repository pada drive D:.

     ```
     user@SELPIYA MINGW64 ~
     $ cd /d/GitHub
     ```

   - Kemudian repository GitHub di-*clone* menggunakan:

     ```
     git clone https://github.com/Selpiya/latihan-git-3224007.git
     ```

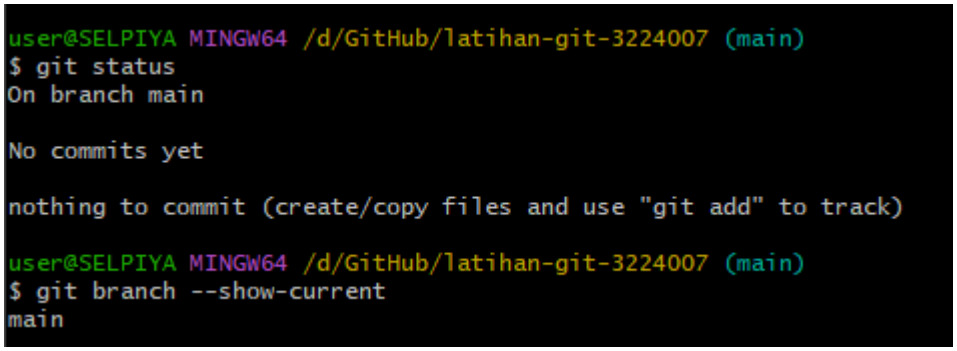
     ```
     user@SELPIYA MINGW64 /d/GitHub
     $ git clone https://github.com/Selpiya/latihan-git-3224007.git
     Cloning into 'latihan-git-3224007'...
     warning: You appear to have cloned an empty repository.
     ```

   - Setelah proses clone selesai, saya masuk folder repository:

     ```
     user@SELPIYA MINGW64 /d/GitHub
     $ cd latihan-git-3224007
     ```

- Untuk memastikan bahwa posisi repository dan branch sudah benar, saya melakukan pengecekan:

```
git status
git branch --show-current
```



Hasil pengecekan menunjukan bahwa repository lokal berada pada:

```
/d/GitHub/latihan-git-3224007
```

Dan branch yang aktif adalah:

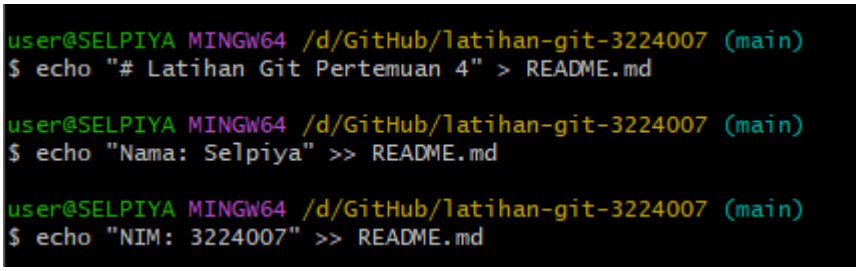
```
main
```

Command git clone berfungsi untuk membuat salinan repository dari GitHub atau remote repository ke komputer lokal. Proses clone ini memungkinkan project dikelola menggunakan Git melalui komputer lokal.

3. Membuat file README.md yang berisi nama dan NIM

Setelah repository berhasil di-*clone*, saya membuat file README.md melalui Git Bash. Command yang digunakan adalah:

```
echo "# Latihan Git Pertemuan 4" > README.md
echo "Nama: Selpiya" >> README.md
echo "NIM: 3224007" >> README.md
```



Command pertama menggunakan tanda > untuk membuat file README.md sekaligus menuliskan judul. Selanjutnya, tanda >> digunakan untuk menambahkan informasi nama dan NIM tanpa menghapus isi yang sebelumnya sudah ada.

Kemudian isi file diperiksa mengunakan:

```
cat README.md
```

Hasilnya:

```
user@SELPIYA MINGW64 /d/GitHub/latihan-git-3224007 (main)
$ cat README.md
# Latihan Git Pertemuan 4
Nama: Selpiya
NIM: 3224007
```

Setelahh itu, saya melakukan pengecekan status repository:

```
git status
```

Pada tahap ini, *README.md* muncul pada bagian **Untracked files**, yang berarti file tersebut sudah tersedia pada working directory tetapi belum dimasukkan ke staging area.

4. **Melakukan commit README.md dengan pesan "docs: add README"**

Sebelum commit dilakukan, file README.md terlebih dahulu ditambahkan ke staging area menggunakan:

```
git add README.md
```

```
user@SELPIYA MINGW64 /d/GitHub/latihan-git-3224007 (main)
$ git add README.md
warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time Git tou
ches it
```

Kemudian dilakukan pengecekan:

```
git status
```

Setelah git add, status file berubah menjadi:

```
user@SELPIYA MINGW64 /d/GitHub/latihan-git-3224007 (main)
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   README.md
```

Hal tersebut menunjukkan bahwa README.md sudah berada di staging area dan siap untuk di-*commit*.

Selanjutnya saya melakukan commit menggunakan pesan yang telah ditentukan pada soal:

```
git commit -m "docs: add README"
```

Commit berhasil dibuat dengan pesan:

```
user@SELPIYA MINGW64 /d/GitHub/latihan-git-3224007 (main)
$ git commit -m "docs: add README"
[main (root-commit) 06e55e3] docs: add README
 1 file changed, 3 insertions(+)
 create mode 100644 README.md
```

Setelah commit, saya kembali melakukan pengecekan:

```
git status
```

Dan diperoleh kondisi:

```
user@SELPIYA MINGW64 /d/GitHub/latihan-git-3224007 (main)
$ git status
On branch main
Your branch is based on 'origin/main', but the upstream is gone.
  (use "git branch --unset-upstream" to fixup)

nothing to commit, working tree clean
```

Artinya, perubahan pada README.md sudah berhasil disimpan ke riwayat repository Git lokal.

5. **Push repository ke GitHub**

Setelah commit berhasil dibuat, perubahan pada branch *main* dikirim ke GitHub menggunakan:

```
git push -u origin main
```

Pada praktik ini digunakan opsi *-u* atau *--set-upstream* karena branch lokal *main* perlu dihubungkan dengan branch remote *origin/main.*

Setelah proses push berhasil, file *README.md* dan commit:

```
user@SELPIYA MINGW64 /d/GitHub/latihan-git-3224007 (main)
$ git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 4 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 272 bytes | 136.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Selpiya/latihan-git-3224007.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Secara umum, *git push* berfungsi untuk mengirim commit dari repository lokal menuju remote repository.

6. **Membuat branch baru dengan nama feature-test**

Branch baru di buat menggunakan:

```
git branch feature-test
```

Untuk memeriksa daftar branch yang tersedia digunakan:

```
git branch
```

Pada tahap tersebut hasilnya menunjukan:

```
user@SELPIYA MINGW64 /d/GitHub/latihan-git-3224007 (main)
$ git branch
  feature-test
* main
```

Tanda (*) menunjukkan bahwa branch aktif masih main, sedangkan branch feature-test sudah berhasil dibuat.

Penggunaan branch memungkinkan pengembangan atau perubahan dilakukan secara terpisah tanpa langsung memengaruhi branch utama.

7. **Berpindah ke branch feature-test**

Setelah branch dibuat, saya berpindah dari branch main ke feature-test dengan:

```
git checkout feature-test
```

Kemudian saya memastikan branch aktif dengan:

```
git branch
```

Hasilnya:

```
user@SELPIYA MINGW64 /d/GitHub/latihan-git-3224007 (feature-test)
$ git branch
* feature-test
  main
```

Saya juga melakukan pengecekan menggunakan:

```
git status
```

Dan diperoleh:

```
user@SELPIYA MINGW64 /d/GitHub/latihan-git-3224007 (feature-test)
$ git status
On branch feature-test
nothing to commit, working tree clean
```

Hal tersebut menunjukan bahwa perpindahan branch berhasil. git checkout digunakan untuk berpindah dari satu branch ke branch lainnya

8. **Membuat file test.txt dan melakukan commit**

Pada branch feature-test, saya membuat file test.txt menggunakan:

```
echo "File test untuk branch feature-test" > test.txt
```

Kemudian isi file diperiksa menggunakan:

```
cat test.txt
```

Hasilnya:

```
user@SELPIYA MINGW64 /d/GitHub/latihan-git-3224007 (feature-test)
$ cat test.txt
File test untuk branch feature-test
```

Selanjutnya dilakukan pengecekan:

```
git status
```

File *test.txt* muncul sebagai:

```
user@SELPIYA MINGW64 /d/GitHub/latihan-git-3224007 (feature-test)
$ git status
On branch feature-test
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        test.txt

nothing added to commit but untracked files present (use "git add" to track)
```

Kemudian file tersebut dimasukkan ke staging area dengan:

```
git add test.txt
```

Setelah itu dilakukan pengecekan:

```
git status
```

Hasilnya berubah menjadi:

```
user@SELPIYA MINGW64 /d/GitHub/latihan-git-3224007 (feature-test)
$ git status
On branch feature-test
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   test.txt
```

Setelah file masuk staging area, dilakukan commit seusai pesan yang ditentukan pada soal:

```
git commit -m "test: add test file"
```

Commit berhasil dibuat dengan pesan:

```
user@SELPIYA MINGW64 /d/GitHub/latihan-git-3224007 (feature-test)
$ git commit -m "test: add test file"
[feature-test 0d01b1f] test: add test file
 1 file changed, 1 insertion(+)
 create mode 100644 test.txt
```

Kemudian dilakukan pengecekan terakhir:

```
git status
```

Dengan hasil:

```
user@SELPIYA MINGW64 /d/GitHub/latihan-git-3224007 (feature-test)
$ git status
On branch feature-test
nothing to commit, working tree clean
```

Artinya file *test.txt* sudah berhasil disimpan dalam riwayat commit pada branch *feature-test.*

9. Push branch feature-test ke GitHub

Setelah commit pada branch feature-test selesai, branch tersebut dikirim ke GitHub menggunakan:

```
git push -u origin feature-test
```

Opsi *-u* digunakan agar branch lokal feature-test terhubung dengan branch remote:

```
user@SELPIYA MINGW64 /d/GitHub/latihan-git-3224007 (feature-test)
$ git push -u origin feature-test
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 4 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 313 bytes | 313.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature-test' on GitHub by visiting:
remote:      https://github.com/Selpiya/latihan-git-3224007/pull/new/feature-test
remote:
To https://github.com/Selpiya/latihan-git-3224007.git
 * [new branch]      feature-test -> feature-test
branch 'feature-test' set up to track 'origin/feature-test'.
```

Setlah push berhasil, repository GitHub mempunyai dua branch, yitu

- *main*
- *feauture-test*

Pada tahap ini branch *feature-test* memiliki satu commit yang belum terdapat pada *main*, yaitu:

```
test: add test file
```

Dengan demikian, branch baru tidak hanya dibuat di repository lokal, tetapi juga sudah tersedia pada repository GitHub.

10. Merge branch feature-test ke main

Sebelum melakukan merge, saya terlebih dahulu kembali ke branch main dengan:

```
git checkout main
```

Kemudian saya memastikan posisi branch aktif menggunakan:

```
git branch
```

Dan diperoleh:

```
user@SELPIYA MINGW64 /d/GitHub/latihan-git-3224007 (main)
$ git branch
  feature-test
* main
```

Setelah dipastikan berada di branch main, branch feature-test digabungkan menggunakan:

```
git merge feature-test
```

Proses merge berhasil dengan mekanisme **fast-forward**, sehingga perubahan yang terdapat pada feature-test, yaitu file test.txt, masuk ke branch main.

```
user@SELPIYA MINGW64 /d/GitHub/latihan-git-3224007 (main)
$ git merge feature-test
Updating 06e55e3..0d01b1f
Fast-forward
 test.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 test.txt
```

Setelah merge, saya melakukan pengecekan dengan:

```
git status
```

Dan:

```
ls
```

Hasil dari *ls* menunjukkan bahawa branch *main* sekarang memiliki:

```
user@SELPIYA MINGW64 /d/GitHub/latihan-git-3224007 (main)
$ ls
README.md  test.txt
```

Karena hasil merge tersebut baru terdapat pada repository lokal, perubahan kemudian dikirim kembali ke GitHub menggunakan:

```
git push origin main
```

```
user@SELPIYA MINGW64 /d/GitHub/latihan-git-3224007 (main)
$ git push origin main
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Selpiya/latihan-git-3224007.git
   06e55e3..0d01b1f  main -> main
```

Setelah proses push selesai, status repository diperiksa kembali:

```
git status
```

Hasil akhirnya menunjukkan:

```
user@SELPIYA MINGW64 /d/GitHub/latihan-git-3224007 (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

Untuk melihat riwayat commit, saya menggunakan:

```
git log --oneline
```

Riwayat commit menunjukkan dua commit utama, yaitu

```
user@SELPIYA MINGW64 /d/GitHub/latihan-git-3224007 (main)
$ git log --oneline
0d01b1f (HEAD -> main, origin/main, origin/feature-test, feature-test) test: add test file
06e55e3 docs: add README
```

git merge berfungsi untuk menggabungkan perubahan dari branch lain ke branch yang sedang aktif. Pada praktik ini, perubahan dari feature-test digabungkan ke main.

Hasil Akhir Praktik

Setelah seluruh proses selesai, repository GitHub latihan-git-3224007 memiliki dua branch, yaitu main dan feature-test. Branch main sudah berisi file README.md dan test.txt, serta mempunyai dua commit utama, yaitu docs: add README dan test: add test file.

Hal ini menunjukkan bahwa proses:

clone → membuat file → staging → commit → push → branching → checkout → commit pada branch → push branch → merge → push hasil merge

Telah berhasil dilakukan.

Link Repository GitHub : https://github.com/Selpiya/latihan-git-3224007.git

Bagian B: Unit Testing Framework

Soal 3 - Konversi Test Case ke Kode Unit Test

Method *validasiPendaftaran()* melakukan validasi secara berurutan terhadap nama, umur, email, dan persetujuan syarat. Apabila suatu kondisi tidak terpenuhi, method langsung mengembalikan pesan kesalahan sehingga kondisi setelahnya tidak lagi diperiksa.

Pada analisis White-box Testing di Tugas Minggu 3, method ini mempunyai beberapa jalur independen berdasarkan kondisi P1 sampai P5. Hasil analisis saya pada Minggu 3 memperoleh **Cyclomatic Complexity sebesar 6**, karena kondisi email *email == null || !email.contains("@")* dipisahkan menjadi dua predicate, yaitu P3 dan P4. Oleh karena itu, terdapat enam independent path pada analisis sebelumnya.

Namun sesuai instruksi pada Soal 3 Pertemuan 4 yang meminta **5 test case**, maka lima test case utama berikut dipilih dari independent path yang telah dianalisis sebelumnya. Setiap input dirancang agar kondisi sebelum path yang ingin diuji tetap valid sehingga eksekusi benar-benar mencapai kondisi yang menjadi target pengujian.

1. Lima Test Case Berdasarkan Independent Path

Test Case	Referensi Path Minggu 3	Input	Expected Output	Kondisi yang Diuji
TC-01	Path 1	nama=null, umur=20, email="budi@mail.com", setujuSyarat=true	"Nama harus diisi"	P1 true: nama bernilai null
TC-02	Path 2	nama="Budi", umur=16, email="budi@mail.com", setujuSyarat=true	“Umur minimal 17 tahun”	P1 false, P2 true: umur kurang dari 17
TC-03	Path 4	nama="Budi", umur=20, email="budimail.com", setujuSyarat=true	“Email tidak valid”	P1–P3 false, P4 true: email tidak mengandung @
TC-04	Path 5	nama="Budi", umur=20, email="budi@mail.com", setujuSyarat=false	“Harus menyetujui syarat dan ketentuan”	P1–P4 false, P5 true

TC-05	Path 6	nama="Budi", umur=20, email="budi@mail.com", setujuSyarat=true	“Pendaftaran berhasil”	Seluruh validasi lolos

Penjelasan masing-masing test case

1) Test Case 1- Nama tidak diisi

Input:

nama = null
umur = 20
email = "budi@mail.com"
setujuSyarat = true

Expected output:

“Nama harus diisi”

Test case pertama menguji kondiis P1:

```
if (nama == null || nama.isEmpty())
```

Karena *nama* bernilai *null*, kondisi tersebut bernilai *true*. Method langsung mengembalikan: *Nama harus diisi***.** Dan kondisi umur, email, serta persetujuan syarat tidak lagi diperiksa.

Jalur berdasarkan analisis Minggu 3 adalah:

1 → 2 → 11

2) Test Case 2 – Umur kurang dari 17 tahun

Input:

nama = "Budi"
umur = 16
email = "budi@mail.com"
setujuSyarat = true

Expected output:

“Umur minimal 17 tahun”

Pada test case ini, kondisi nama dibuat valid supaya P1 bernilai false dan proses dapat dilanjutkan ke P2:

```
if (umur < 17)
```

Karena nilai umur adalah 16, maka *16 < 17* bernilai true sehingga method mengembalikan: *Umur minimal 17 tahun*

Jalurnya adalah:

1 → 3 → 4 → 11

Path tersebut sama dengan path 2 yang dibuat pada Tugas Minggu 3

3) Test Case 3 – Email tidak mengandung karakter @

Input:

nama = "Budi"
umur = 20
email = "budimail.com"
setujuSyarat = true

Expected output:

"Email tidak valid"

Pada test case ini, nama dan umur dibuat valid agar eksekusi dapat mencapai pemeriksaan email:

```
if (email == null || !email.contains("@"))
```

Dengan input email = "budimail.com", evaluasi kondinya adalah

```
email == null                    → false
!email.contains("@")             → true
```

Karena email tidak mengandung karakter **@,** kondisi email bernilai true sehingga method mengembalikan: *Email tidak valid.*

Jalur seusai Path 4 pada analisis Minggu 3:

1 → 3 → 5 → 6 → 7 → 11

4) **Test Case 4 – Syarat dan ketentuan tidak disetujui**

Input:

```
nama = "Budi"
umur = 20
email = "budi@mail.com"
setujuSyarat = false
```

Expected output:

```
“Harus menyetujui syarat dan ketentuan”
```

Nama umur, dan email sengaja dibuat valid agar eksekusi mencapai kondisi:

```
if (!setujuSyarat)
```

Karena *setujuSyarat*=false, *maka:*

```
!setujuSyarat
```

Menghasilkan *true.* Method kemudian mengembalikan: *Harus menyetujui syarat dan ketentuan*

Jalur seusai Path 5 pada analisis Minggu 3:

```
1 → 3 → 5 → 6 → 8 → 9 → 11
```

Yang merupakan Path 5 pada Tugas Minggu 3

5) **Test Case 5 – Seluruh data valid**

Input:

```
nama = "Budi"
umur = 20
email = "budi@mail.com"
setujuSyarat = true
```

Expected output:

```
“Pendaftaran berhasil”
```

Test case terakhir digunakan untuk menguji **jalur sukses**, yaitu ketika seluruh kondisi validasi tidak menghasilkan kesalahan.

Evaluasinya:

```
nama == null / kosong       → false

umur < 17                   → false

email == null               → false

!email.contains("@")        → false

!setujuSyarat               → false
```

Karena seluruh validasi lolos, statement terakhir dieksekusi:

```
return "Pendaftaran berhasil";
```

Jalur adalah:

```
1 → 3 → 5 → 6 → 8 → 10 → 11
```

Dan merupakan Path 6 dari analisis Minggu 3

2. Implementasi Test Case ke Kode Unit Test Junit

Implementasi unit test menggunakan **JUnit 5**. Materi Pertemuan 4 menjelaskan bahwa JUnit 5 menggunakan anotasi seperti *@Test* dan *@BeforeEach*, serta assertion seperti *assertEquals()* untuk memverifikasi hasil yang diperoleh dengan nilai yang diharapkan.

Penulisan unit test juga mengikuti pola Arrange-Act-Assert (AAA):

- **Arrange**: menyiapkan objek dan data input.
- **Act**: menjalankan method yang diuji.
- **Assert**: membandingkan hasil aktual dengan expected output.

Karena potongan kode pada soal hanya menampilkan method dan tidak menyebutkan nama class secara eksplisit, pada implementasi berikut diasumsikan method *validasiPendaftaran()* berada di dalam class:

```
ValidasiPendaftaran
```

Kode Junit 5

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

public class ValidasiPendaftaranTest {

    private ValidasiPendaftaran validator;

    @BeforeEach
    public void setUp() {
        validator = new ValidasiPendaftaran();
    }

    // TC-01 / Path 1
    // Menguji nama bernilai null
    @Test
    public void testPath1_NamaNull() {

        // Arrange
        String nama = null;
        int umur = 20;
        String email = "budi@mail.com";
        boolean setujuSyarat = true;

        // Act
        String result = validator.validasiPendaftaran(
                nama, umur, email, setujuSyarat
        );

        // Assert
        assertEquals("Nama harus diisi", result);
    }
```

```
    // TC-02 / Path 2
    // Menguji umur kurang dari 17 tahun
    @Test
    public void testPath2_UmurKurangDari17() {

        // Arrange
        String nama = "Budi";
        int umur = 16;
        String email = "budi@mail.com";
        boolean setujuSyarat = true;

        // Act
        String result = validator.validasiPendaftaran(
                nama, umur, email, setujuSyarat
        );

        // Assert
        assertEquals("Umur minimal 17 tahun", result);
    }

    // TC-03 / Path 4
    // Menguji email yang tidak memiliki karakter @
    @Test
    public void testPath4_EmailTanpaAt() {

        // Arrange
        String nama = "Budi";
        int umur = 20;
        String email = "budimail.com";
        boolean setujuSyarat = true;

        // Act
        String result = validator.validasiPendaftaran(
                nama, umur, email, setujuSyarat
        );
```

```
        // Assert
        assertEquals("Email tidak valid", result);
    }

    // TC-04 / Path 5
    // Menguji syarat dan ketentuan tidak disetujui
    @Test
    public void testPath5_SyaratTidakDisetujui() {

        // Arrange
        String nama = "Budi";
        int umur = 20;
        String email = "budi@mail.com";
        boolean setujuSyarat = false;

        // Act
        String result = validator.validasiPendaftaran(
                nama, umur, email, setujuSyarat
        );

        // Assert
        assertEquals(
                "Harus menyetujui syarat dan ketentuan",
                result
        );
    }

    // TC-05 / Path 6
    // Menguji seluruh data valid
    @Test
    public void testPath6_PendaftaranBerhasil() {

        // Arrange
        String nama = "Budi";
```

```
        int umur = 20;
        String email = "budi@mail.com";
        boolean setujuSyarat = true;

        // Act
        String result = validator.validasiPendaftaran(
                nama, umur, email, setujuSyarat
        );

        // Assert
        assertEquals("Pendaftaran berhasil", result);
    }
}
```

Hubungan Test Case dengan Independent Path

Supaya hubungan antara tugas Minggu 3 dan implementasi JUnit terlihat jelas, pemetaannya dapat ditampilkan sebagai berikut:

Test Method	Independent Path Minggu 3	Kondisi Utama	Expected
testPath1_NamaNull()	Path 1	P1 true	“Nama harus diisi”
testPath2_UmurKurangDari17()	Path 2	P2 true	“Umur minimal 17 tahun”
testPath4_EmailTanpaAt()	Path 4	P4 true	“Email tidak valid”
testPath5_SyaratTidakDisetujui()	Path 5	P5 true	“Harys menyetujui syarat dan ketentuan”
testPath6_PendaftaranBerhasil()	Path 6	Semua kondisi valid	“Pendaftaran berhasil”

Dengan demikian, setiap test case mempunyai hubungan langsung dengan jalur kontrol yang sebelumnya telah dianalisis menggunakan White-box Testing. Hal ini sesuai dengan materi Pertemuan 4 yang menjelaskan bahwa test case dari independent path pada Minggu 3 kemudian dikonversi menjadi method *@Test* di JUnit.

Test tambahan untuk Path 3

```
@Test
public void testTambahan_EmailNull() {

    // Arrange
    String nama = "Budi";
    int umur = 20;
    String email = null;
    boolean setujuSyarat = true;

    // Act
    String result = validator.validasiPendaftaran(
            nama, umur, email, setujuSyarat
    );

    // Assert
    assertEquals("Email tidak valid", result);
}
```

Test in menguji:

1 → 3 → 5 → 7 → 11

Yaitu saat *email == null* bernilai true sehingga Java melakukan **short-circuit evaluation** dan tidak mengevaluasi *email.contains("@")*.

Soal 4 – Implementasi Unit Test pada Class StudentGrade

Kode yang diuji terdiri atas dua method, yaitu

- *calculateGrade(int score),* yang mengubah nilai numerik menjadi grade A sampai E serta menolak nilai di luar rentang 0–100.
- *isPassed(int score)*, yang menentukan apakah mahasiswa lulus dengan batas minimum nilai 60 serta juga menolak nilai di luar rentang 0–100.

1. Menghitung Cylomatic Complexity Method

calculateGrade()

Cylomatic Complexity digunakan untuk mengukur jumlah jalur independen yang terdapat dalam alur kontrol program. Secara praktis, nilai Cyclomatic Complexity juga menunjukkan jumlah minimum *independent path* yang perlu diuji agar struktur logika utama program dapat tercakup.

Pada method *calculateGrade(),* terdapat kondisi berikut:

```
if (score < 0 || score > 100)
```

Karena Java menggunakan evaluasi *short-circuit* pada operator | | Pada analisis ini digunakan pemodelan short-circuit evaluation, sehingga dua kondisi atomik score < 0 dan score > 100 pada ekspresi || diperlakukan sebagai decision node yang terpisah. Kondisi tersebut dapat dipandang sebagai dua predicate:

- **P1:** *score < 0*
- **P2:** *score > 100*

Kemudian terdapat empat decision berikutnya:

- **P3:** *score >= 80*
- **P4:** *score >= 70*
- **P5:** *score >= 60*
- **P6:** *score >= 50*

Dengan demikian, terdapat 6 predicate/ decision.

Perhitungan dengan rumus Predicate + 1

Rumus:

```
V(G) = P + 1
```

Dengan:

P = 6, Maka:

```
V(G) = P + 1

V(G) = 7
```

Jadi, Cyclomatic Complexity dari method *calculateGrade()* adalah 7. Artinya terdapat 7 **independent path** yang secara struktural perlu diperhatikan dalam pengujian.

Pengujian dengan rumus Edge dan Node

Agar perhitungannya tidak hanya bergantung pada satu rumus, alur method dapat dipetakan sebagai berikut.

Node	Statement/ Kondisi	Keterangan
1	*Score < 0*	P1
2	*Throw IllegalArgumentException*	Input tidak valid
3	*Score > 100*	P2
4	*Score >= 80*	P3
5	R*eturn "A"*	Grade A
6	*Score >= 70*	P4
7	Return "B"	Grade B
8	Score >= 60	P5
9	Return "C"	Grade C
10	Score >= 50	P6
11	Return "D"	Grade D
12	Return "E"	Grade E
13	Exit	Node keluar

Jumlah Node: **N = 13**

Jumlah edge : **E = 18**

Karbea hanya terdapat satu connected component: **P = 1**

Rumus:

```
V(G) = E - N + 2P
```

Sehingga:

```
V(G) = 18 - 13 + 2(1)

V(G) = 18 - 13 + 2

V(G) = 7
```

Hasilnya sama dengan perhitungan Predicate + 1, sehingga nilai Cyclomatic Complexity konsisten.

2. **Menentukan Independent Path Method**

   calculateGrade()

   Berdasarkan nilai Cyclomatic Complexity sebesar 7, maka terdapat tujuh independent path. Saya menggunakan nilai pada batas kategori agar pengujiannya tidak hanya melewati jalur program, tetapi juga sekaligus memeriksa nilai batas yang menentukan perubahan hasil grade.

   Path 1 – Score di bawah 0

   Kondisi:

   ```
   P1 = true
   ```

   Conoh input:

   ```
   Score = -1
   ```

   Alur:

   ```
   1 → 2 → 13
   ```

Output:

```
IllegalArgumentException
```

Dengan pesan:

```
Score must be between 0 and 100
```

Path ini menguji batas bawah input ketika kurang dari 0

Path 2 – Score di atas 100

Kondisi:

```
P1 = false

P2 = true
```

Contoh input:

```
Score = 101
```

Alur:

```
1 → 3 → 2 → 13
```

Output:

```
IllegalArgumentException
```

Path ini berbeda dari path 1 karena *score < 0* bernilai false sehingga evaluasi dilanjutkan ke kondisi kedua, yaitu *score > 100*

Path 3 – Grade A

Kondisi:

```
P1 = false

P2 = false

P3 = true
```

Contoh input:

```
Score = 80
```

Alur:

```
1 → 3 → 4 → 5 → 13
```

Expected Output:

```
"A"
```

Nilai 80 dipilih karena merupakan batas minimum grade A

Path 4 – Grade B

Kondisi:

```
P1 = false

P2 = false

P3 = false

P4 = true
```

Contoh input:

```
Score = 70
```

Alur:

```
1 → 3 → 4 → 6 → 7 → 13
```

Expected Output:

```
"B"
```

Nilai 70 merupakan batas minimum grade B

Path 5 – Grade C

Kondisi:

```
P1 = false

P2 = false

P3 = false

P4 = false

P5 = true
```

Contoh input:

```
Score = 60
```

Alur:

```
1 → 3 → 4 → 6 → 8 → 9 → 13
```

Expected Output:

```
"C"
```

Method 60 dipilih karena merupakan batas minimum grade C sekaligus batas kelulusan pada method *isPassed()*.

Path 6 – Grade D

Kondisi:

```
P1 = false

P2 = false

P3 = false

P4 = false

P5 = false

P6 = true
```

Contoh input:

```
Score = 50
```

Alur:

```
1 → 3 → 4 → 6 → 8 → 10 → 11 → 13
```

Expected Output:

```
"D"
```

Nilai 50 merupakan batas minimum grade D.

Path 7 – Grade E

Kondisi:

```
P1 = false

P2 = false

P3 = false

P4 = false

P5 = false

P6 = false
```

Contoh input:

```
Score = 49
```

Alur:

```
1 → 3 → 4 → 6 → 8 → 10 → 12 → 13
```

Expected Output:

```
"E"
```

Nilai 49 dipilih karena tepat satu angka di bawah batas minimum grade D.

3. Implementasi Unit Junit untuk calculateGrade() dan isPassed()

A. Test Case untuk *calculateGrade()*

Test	Input	Expected Result	Tujuan
TC1	-1	Exception	Score di bawah batas minimum
TC2	101	Exception	Score melebihi batas maksimum
TC3	80	“A”	Batas minimum grade A
TC4	70	“B”	Batas minimum grade B
TC5	60	“C”	Batas minimum grade C
TC6	50	“D”	Batas minimum grade D
TC7	49	“E”	Tepat di bawah batas grade D

B. Test Case untuk i*sPassed()*

Method:

```
public boolean isPassed(int score) {
    if (score < 0 || score > 100) {
        throw new IllegalArgumentException(
            "Score must be between 0 and 100"
        );
    }

    return score >= 60;
}
```

Untuk method ini, nilai yang paling penting adalah batas valid input dan batas kelulusan. Saya menggunakan:

Test	Input	Expected Result	Tujuan
TC8	-1	Exception	Di bawah batas valid
TC9	101	Exception	Di atas batas valid
TC10	59	false	Te[at di bawah batas lulus
TC11	60	true	Tepat pada batas minimum lulus
TC12	0	false	Batas minimum score valid
TC13	100	true	Batas maksimum score valid

Pengujian 59 dan 60 menjadi penting karena keduanya berada tepat di sekitar batas:

```
Score >= 60
```

Sehingga dapat memastikan bahawa kondisi kelulusan tidak mengalami kesalahan *off-by-one.*

Implementasi Junit 5

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

public class StudentGradeTest {

    private StudentGrade studentGrade;

    @BeforeEach
    public void setUp() {
        studentGrade = new StudentGrade();
    }

    // ============================================================
    // TEST calculateGrade()
```

```
// ============================================================

@Test
public void testCalculateGrade_ScoreBelowZero() {
    int score = -1;

    IllegalArgumentException exception = assertThrows(
            IllegalArgumentException.class,
            () -> studentGrade.calculateGrade(score)
    );

    assertEquals(
            "Score must be between 0 and 100",
            exception.getMessage()
    );
}

@Test
public void testCalculateGrade_ScoreAbove100() {
    int score = 101;

    IllegalArgumentException exception = assertThrows(
            IllegalArgumentException.class,
            () -> studentGrade.calculateGrade(score)
    );

    assertEquals(
            "Score must be between 0 and 100",
            exception.getMessage()
    );
}

@Test
public void testCalculateGrade_GradeA() {
    int score = 80;
```

```
    String result = studentGrade.calculateGrade(score);

    assertEquals("A", result);
}

@Test
public void testCalculateGrade_GradeB() {
    int score = 70;

    String result = studentGrade.calculateGrade(score);

    assertEquals("B", result);
}

@Test
public void testCalculateGrade_GradeC() {
    int score = 60;

    String result = studentGrade.calculateGrade(score);

    assertEquals("C", result);
}

@Test
public void testCalculateGrade_GradeD() {
    int score = 50;

    String result = studentGrade.calculateGrade(score);

    assertEquals("D", result);
}

@Test
public void testCalculateGrade_GradeE() {
```

```
    int score = 49;

    String result = studentGrade.calculateGrade(score);

    assertEquals("E", result);
}

// ============================================================
// TEST isPassed()
// ============================================================

@Test
public void testIsPassed_ScoreBelowZero() {
    int score = -1;

    IllegalArgumentException exception = assertThrows(
            IllegalArgumentException.class,
            () -> studentGrade.isPassed(score)
    );

    assertEquals(
            "Score must be between 0 and 100",
            exception.getMessage()
    );
}

@Test
public void testIsPassed_ScoreAbove100() {
    int score = 101;

    IllegalArgumentException exception = assertThrows(
            IllegalArgumentException.class,
            () -> studentGrade.isPassed(score)
    );
```

```
    assertEquals(
            "Score must be between 0 and 100",
            exception.getMessage()
    );
}

@Test
public void testIsPassed_Score59_ReturnsFalse() {
    int score = 59;

    boolean result = studentGrade.isPassed(score);

    assertFalse(result);
}

@Test
public void testIsPassed_Score60_ReturnsTrue() {
    int score = 60;

    boolean result = studentGrade.isPassed(score);

    assertTrue(result);
}

@Test
public void testIsPassed_ScoreZero_ReturnsFalse() {
    int score = 0;

    boolean result = studentGrade.isPassed(score);

    assertFalse(result);
}

@Test
public void testIsPassed_Score100_ReturnsTrue() {
```

```
        int score = 100;

        boolean result = studentGrade.isPassed(score);

        assertTrue(result);
    }
}
```

Bagian C: GitHub Actions

Soal – 5 Analisis Workflow YAML

1. **Apa nama workflow di atas?**

   Nama workflow tersebut adalah:

   ```
   Name: Python CI
   ```

   Jadi, nama workflow-nya adalah *Python CI.*

   Pada Github Actions, bagian name digunakan untuk menentukan nama workflow yang akan ditampilkan pada tab Actions di repository GitHub. Dengan demikian, ketika workflow ini dijalankan, pengguna dapat mengenalinya sebagai workflow Python CI.

2. **Trigger/event apa saja yang memicu workflow?**

   Pada workflow terdapat konfihurasi:

   ```
   on:
      push:
          branches: [ main, develop ]


     pull_request:
        branches: [ main ]
   ```

   Berdasarkan konfigurasi tersebut, terdapat dua jenis event utama yang dapat memicu workflow.

   a. **Event *push***

      Workflow akan dijalankan ketika terjadi push ke branch:

      - `Main`
      - `Develop`

      Artinya, apabila developer mengirim commit ke branch main atau develop, GitHub Actions akan secara otomatis menjalankan workflow Python CI.

      Contohnya:

      ```
      git push origin main
      ```

      Atau:

      ```
      git push origin develop
      ```

      Akan memicu workflow.

Namun, apabila push dilakukan ke branch lain seperti:

- Feature – login
- Feature – test
- Bugfix

Workflow ini tidak akan berjalan melalui trigeer push, karena branch tersebut tidak tercantum pada:

```
Branches: [ main, develop ]
```

b. Event pull_request

Workflow juga akan berjalan ketika terdapat pull request yang diarahkan ke branch *main.*

Konfigurasinya adalah:

```
Pull_request:
  Branches: [main]
```

Artinya, misalnya terdapat branch:

```
Feature – login
```

Kemudian dibuat Pull Request:

```
Feature-login → main
```

Maka workflow *Python CI* akan dijalankan.

Tujuannya adalah memastikan bahwa perubahan yang akan digabungkan ke branch utama sudah melewati proses pengujian terlebih dahulu.

3. Apa fungsi dari Strategy matrix pada workflow di atas?

Bagian workflow:

```
strategy:
  matrix:
    python-version: ['3.10', '3.11', '3.12']
```

Digunakan untuk menjalankan job pengujian pada beberapa versi Python yang berbeda.

Workflow ini, versi yang diuji adalah:

- `Python 3.10`
- `Python 3.11`
- `Python 3.12`

Artinya, GitHub Actions tidak hanya menjalankan tes satu kali, tetapi juga membuat beberapa variasi job berdasarkan nilai yang terdapat pada matrix.

Secara konseptual, GitHub Actions akan membentuk:

- `Job 1 → Python 3.10`
- `Job 2 → Python 3.11`
- `Job 3 → Python 3.12`

Pada masing-masing job, langkah yang sama akan dijalankan, mulai dari checkout repository, setup Python, instalasi dependency, sampai menjalankan test.

Hubungan dengan matrix.python-version

Nilai matrix digunakan pada bagian:

```
- name: Setup Python ${{ matrix.python-version }}
  uses: actions/setup-python@v5
  with:
    python-version: ${{ matrix.python-version }}
```

Ekspresi:

```
${{ matrix.python-version }}
```

Sehingga pada eksekusi pertama dapat menjadi: **Python 3.10**

Pada eksekusi kedua : **3.11**

Dan pada eksekusi ketiga : **3.12**

Mengapa matrix strategy penting?

Menurut saya, fungsi utamanya bukan hanya menjalankan test berulang kali, tetapi memastikan bahwa kode program memiliki **compatibility** yang baik terhadap beberapa versi environment.

Misalnya:

- Python 3.10 → PASS
- Python 3.11 → PASS
- Python 3.12 → FAIL

Maka developer dapat mengetahui bahwa terdapat masalah kompatibilitas pada Python 3.12, meskipun program masih berjalan dengan benar pada versi sebelumnya.

Dengan demikian, manfaat *stategy matrix* pada workflow tersebut adalah:

- menguji program pada beberapa versi Python,
- mendeteksi masalah kompatibilitas,
- menghindari pengujian manual satu per satu,
- meningkatkan keandalan proses Continuous Integration,
- dan memastikan perubahan kode tetap bekerja pada environment yang berbeda.

Jadi, pada workflow ini satu konfigurasi YAML dapat menjalankan pengujian untuk tiga versi Python tanpa perlu membuat tiga workflow terpisah.

4. **Sebutkan 5 step yang ada di dalam job test!**

Pada potongan workflow yang benar-benar ditampilkan pada lembar soal, hanya terdapat **4 steps**, yaitu:

Step 1 – Checkout code

```
- Name: Checkout code
  Uses: actions/ checkout@v4
```

Step ini digunakan untuk mengambil source code dari repository dan menempatkannya ke runner GitHub Actions.

Step 2 – Setup Python

```
- Name: Setup Python %{{ matrix.pyhton-version}}
  Uses: actions/setup-pyhton@v5
  with:
    Pyhton-version: ${{matrix.python-version}}
```

Step ini digunakan untuk menyiapkan environment Python berdasarkan versi yang terdapat pada matrix:

- 3.10
- 3.11
- 3.12

Dengan demikian, setiap job menggunakan versi Python yang sesuai dengan konfigurasi matrix.

Step 3 – Install dependencies

```
- Name: Install dependencies
  run: |
      python -m pip install –
     -upgrade pip pip install pytest pytest-cov
```

Step in digunakan untuk meyiaplan dependency yang diperlukan untuk pengujian.

Command:

```
python -m pip install --upgrade pip
```

Digunakan untuk memperbarui pip.

Sedangkan

```
Pip install pytest-cov
```

Digunakan untuk menginstall:

- *Pytest* sebagai framework testing
- *Pytest-cov* sebagai plugin untuk menghasilkan code coverage.

Step 4 – Run test

```
- Name: Run tests
  run: pytest --cov=src --cov-report=xml
```

Step ini digunakan untuk menjalan seluru test mengunakan *pytest*

Parameter:

```
--cov=src
```

Berarti coverage dihitung terhadap source code yang berada dalam directory *src*

Sedangkan:

```
.--cov-report=xml
```

Berarti laporan coverage akan dibuat dalam format XML. Dengan demikian, step ini tidak hanya menjalankan test, tetapi juga mengukur code coverage.

Catatan Analisis

Menurut saya, terdapat ketidaksesuaian kecil antara potongan YAML pada soal dengan pertanyaan nomor 4.

Potongan YAML pada soal hanya menampilkan:

1) Checkout code
2) Setup Python
3) Install dependencies
4) Run tests

Pada materi, workflow versi lengkap memang memiliki step kelima berupa: **Upload coverage report**. Oleh karena itu, saya membedakan antara empat step yang benar-benar tampak pada YAML soal dan step kelima yang terdapat pada versi workflow lengkap di materi Pertemuan 4. Dengan cara ini, jawaban tidak menambahkan isi yang sebenarnya tidak ada pada potongan soal, tetapi tetap menjelaskan alasan mengapa pertanyaan meminta lima steps.

Soal 6 – Membuat Workflow YAML

Pada soal ini dibuat workflow **GitHub Actions** untuk project Java berbasis Maven. Workflow dikonfigurasi agar pengujian unit dapat dijalankan secara otomatis ketika terjadi push ke branch main atau develop, serta ketika terdapat pull_request yang ditujukan ke branch main. Runner yang digunakan adalah ubuntu-latest, sedangkan environment Java menggunakan **JDK 17**. Setelah unit test selesai dijalankan, workflow menghasilkan laporan code coverage menggunakan **JaCoCo** dan mengunggah hasil laporan tersebut sebagai artifact bernama jacoco-report. Konfigurasi ini mengikuti ketentuan yang diberikan pada soal.

File Workflow YAML

File workflow disimpan dapa:

```
.github/workflows/java-unit-test.yml
```

Isi lengkap file YAML adalah sebagai berikut:

```yaml
name: Java Unit Test


on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up JDK 17
        uses: actions/setup-java@v4
```

```
        with:
          java-version: '17'
          distribution: 'temurin'

      - name: Run tests
        run: mvn test

      - name: Generate coverage report
        run: mvn jacoco: report

      - name: Upload coverage report
        uses: actions/upload-artifact@v4
        with:
          name: jacoco-report
          path: target/site/jacoco/
```

Penjelasan Konfigurasi Workflow

1. **Nama Workflow**

```
Name: Java Unit Test
```

Bagian *name* digunakan untuk menentukan nama workflow yang akan ditampilkan pada tab **Actions** di GitHub. Dalam soal, nama workflow yang diminta adalah **Java Unit Test**. Materi dosen juga menjelaskan bahwa *name* merupakan salah satu komponen utama dalam struktur file workflow GitHub Actions.

2. **Trigger atau Event**

```
    on:
      push:
        branches: [ main, develop ]
      pull_request:
        branches: [ main ]
```

Bagian *on* menentukan event yang akan memicu workflow. Workflow akan dijalankan dalam dua kondisi:

- Ketika terjadi push ke branch main,
- Ketika terjadi push ke branch develop,
- Serta ketika terdapat pull_request yang diarahkan ke branch main.

Dengan konfigurasi tersebut, proses pengujian dapat dijalankan secara otomatis setiap kali terdapat perubahan penting pada branch utama maupun branch pengembangan.

3. Job dan Runner

```
jobs:
  test:
    runs-on: ubuntu-latest
```

Bagian *jobs* mendefinisikan pekerjaan yang akan dijalankan oleh GitHub Actions. Pada workflow ini hanya terdapat satu job bernama:

```
Test
```

Sedangkan:

```
Runs-on: ubuntu-latest
```

Menunjukkan bahwa job akan dijalankan menggunakan runner berbasis Ubuntu yang disediakan oleh GitHub.

Penjelasan Lima Steps pada Job

1) Step 1 – Checkout Code

```
- name: Checkout code
  uses: actions/checkout@v4
```

Step pertama menggunakan **actions/checkout@v4** untuk mengambil source code dari repository GitHub ke dalam runner.

Proses ini diperlukan karena runner GitHub Actions pada awalnya belum memiliki kode project. Setelah checkout dilakukan, file seperti:

- Pom.xml
- Src/main/java/
- Src/test/java/

2) **Step 2 – Set up JDK 17**

```
- name: Set up JDK 17
  uses: actions/setup-java@v4
  with:
      java-version: '17'
      distribution: 'temurin'
```

Step ini digunakan untuk menyimpan enviroment Java pada runner. Action yang digunakan adalah **actions/setup-java@v4** dengan versi Java 17 dan distribusi *Temurin*.

3) **Step 3 – Run Test**

```
- name: Run test
  Run: mvn test
```

Step ketiga menjalankan **mvn test**. Command tersebut digunakan oleh Maven untuk mengompilasi dan menjalankan seluruh unit test yang terdapat pada direktori:

```
Src/test/java
```

Pada project yang saya gunakan dalam praktik, command ini menjalankan unit test JUnit 5 untuk class StudentGrade.

Hasil pengujian yang diperoleh adalah:

```
Tests run: 13, Failures: 0, Errors: 0, Skipped: 0
```

Artinya, seluruh 13 unit test berhasil dijalankan tanpa failure maupun error.

4) **Step 4 – Generate Coverage Report**

```
- name: Generate coverage report
  Run: mvn jacoco: report.
```

Step keempat menjalankan jaCoCo menggunakan:

```
Mvn jacoco: report
```

JaCoCo digunakan untuk menghasilkan laporan code coverage, yaitu laporan yang menunjukkan seberapa banyak kode program telah dieksekusi oleh unit test

Berdasarkan konfigurasi Mavan, laporan JaCoCo dihasilkan pada

```
Target/site/jacoco
```

5) Step 5 – Upload Coverage Report

```
- name: Upload coverage report
  Uses: actions/upload-artifact@v4
  With:
    Name: jacoco-report
    Path: target/site/jacoco
```

Step terakhir digunakan untuk mengunggah hasil laporan JaCoCo sebagai artifact GitHub Actions.

Actions yang digunakan adalah:

```
Actions/upload-artifact@v4
```

Artifact diberi nama

Berdasarkan konfigurasi Maven, laporan JaCoCo dihasilkan pada

```
Jacoco-report
```

Sedangkan file yang diunggah berasal dari:

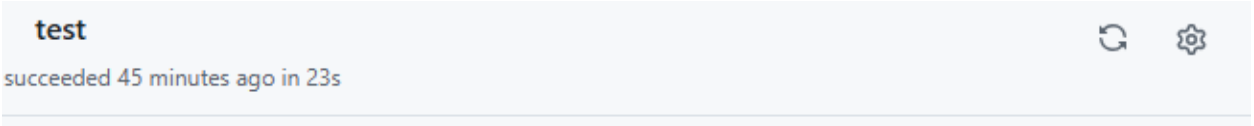
```
Target/side/jacoco
```

Dengan demikian, setelah workflow selesai, laporan coverage tetap dapat diakses dan diunduh melalui halaman workflow GitHub Actions.

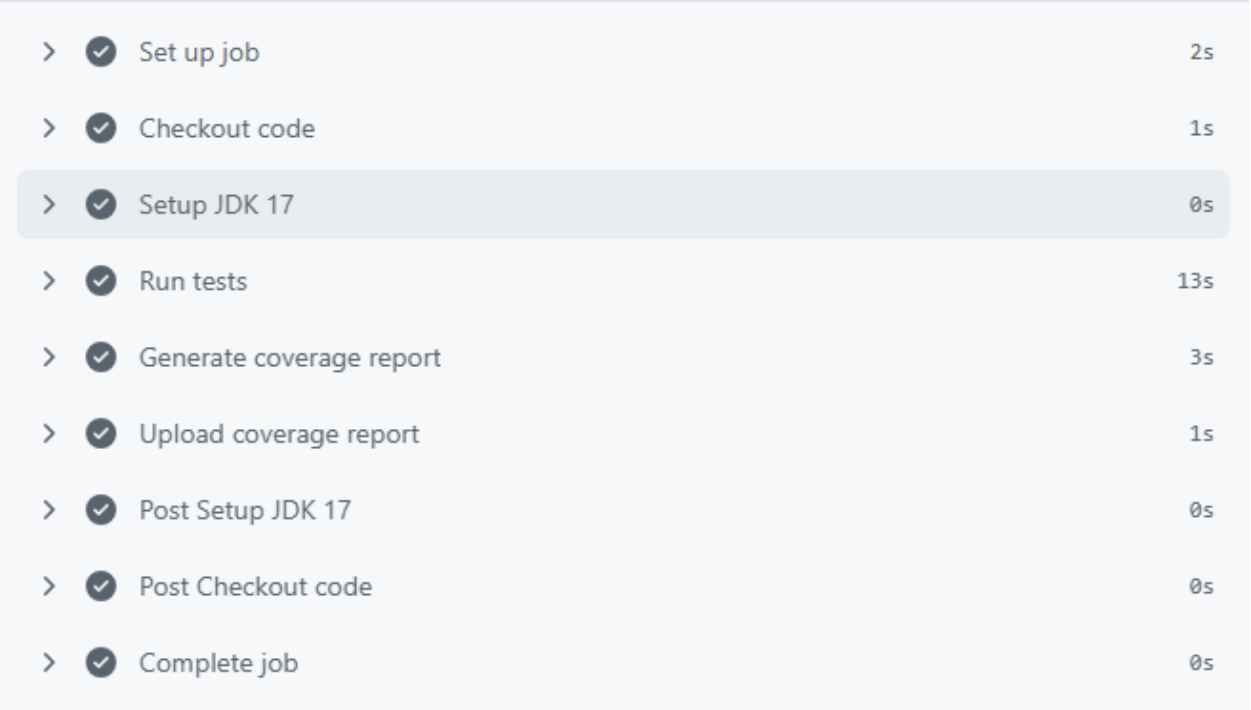
Hasil Implementasi Workflow

Workflow tersebut kemudian saya implementasikan pada repository GitHub dan dijalankan melalui event push ke branch main.

Hasil eksekusi menunjukkan bahwa job:

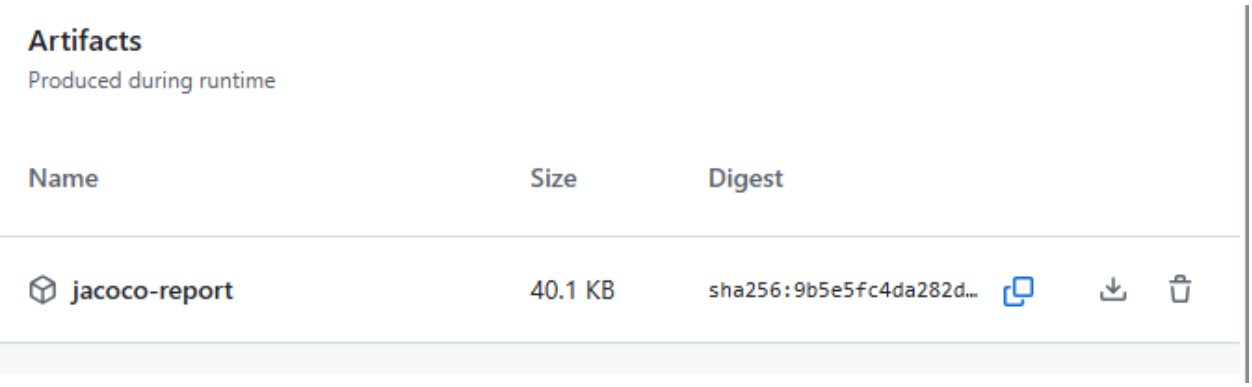


Berstatus **succeeded,** dan sleuruh lima step utama berhasil dijalankan:



GitHub Actions juga menampilkan beberapa step tambahan seperti Set up job, Post Setup JDK 17, Post Checkout code, dan Complete job. Step tersebut merupakan proses internal GitHub Actions, bukan step utama yang dituliskan secara langsung pada file YAML.

Setelah workflow selesai, pada bagian **Artifacts** berhasil dibuat artifact:



Hal tersebut menunjukkan bahwa proses pengujian, pembuatan coverage report, dan upload artifact berhasil dilaksanakan secara otomatis melalui pipeline GitHub Actions.

Repository:

https://github.com/Selpiya/student-grade-ci.git

Bagian D: Code Coverage & Integrasi

Soal 7 – Analisis Code Coverage

Berdasarkan coverage report yang diberikan, project memiliki empat file Python, yaitu calculator.py, login_system.py, payment.py, dan utils.py. Total statement yang tercatat adalah 50 statement, dengan 11 statement tidak tereksekusi, sehingga total coverage project adalah 78%.

1. **Hitung total statement yang ada di project**

   Berdasarkan tabel coverage report, jumlah statement pada setiap file adalah:

   - src/calculator.py = 10 statement
   - src/login_system.py = 15 statement
   - src/payment.py = 20 statement
   - src/utils.py = 5 statement

   Perhitungannya:

   Total Statement
   = 10 + 15 + 20 + 5
   = 50 statements

   Jadi, total statement yang terdapat pada project adalah 50 statement.

   Hasil tersebut juga sesuai dengan bagian TOTAL pada coverage report yang menunjukkan nilai Stmts = 50.

2. **Hitung total statement yang tidak tereksekusi (Miss)**

   Jumlah statement yang tidak tereksekusi pada setiap file adalah:

   - src/calculator.py = 0 statement
   - src/login_system.py = 2 statement
   - src/payment.py = 8 statement
   - src/utils.py = 1 statement

   Perhitungannya:

   Total Miss
   = 0 + 2 + 8 + 1
   = 11 statement

   Jadi, total statement yang tidak tereksekusi adalah 11 statement.

Dengan total 50 statement dan 11 statement tidak tereksekusi, berarti statement yang berhasil dieksekusi oleh test adalah:

Statement tereksekusi
= 50 - 11
= 39 statement

Coverage total dapat diverifikasi dengan perhitungan:

Coverage
= Statement tereksekusi / Total statement × 100%
= 39 / 50 × 100%
= 78%

Hasil tersebut sesuai dengan coverage report yang menunjukkan total coverage sebesar **78%**.

3. **File mana yang memiliki coverage paling rendah? Berapa persen?**

File	Statement	Miss	Coverage
calculator.py	10	0	100%
login_system.py	15	2	87%
payment.py	20	8	60%
utils.py	5	1	80%

File yang memiliki coverage paling rendah adalah **src/payment.py.** Dengan coverage sebesar 60%. Pada file tersebut terdapat 20 statement, tetapi 8 statement tidak pernah dieksekusi oleh test case. Artinya hanya:

20 - 8 = 12 statement

Yang berhasil dieksekusi. Jika dihitung:

12 / 20 × 100% = 60%

Jadi, **file src/payment.py memiliki coverage paling rendah, yaitu 60%**.

4. **Apa yang harus dilakukan untuk meningkatkan coverage pada file payment.py?**

   Untuk meningkatkan coverage pada file payment.py, langkah utama yang harus dilakukan adalah menambahkan test case yang mengeksekusi statement dan jalur program yang saat ini belum tercakup oleh pengujian.

   Pada coverage report terlihat bahwa payment.py mempunyai:

   - 20 statement
   - 8 miss
   - 60% coverage

   Artiya, terdapat 8 statement yang belum pernah dieksekusi oleh test case.

   Beberapa langkah yang dapat dilakukan adalah sebagai berikut.

   a. **Mengidentifikasi statement yang belum tercakup**

      Langkah pertama adalah melihat detail coverage report untuk mengetahui bagian kode mana yang belum pernah dieksekusi.

      Statement yang masih berstatus *miss* harus dianalisis untuk mengetahui kondisi apa yang diperlukan agar statement tersebut dapat dijalankan.

      Dengan demikian, penambahan test case dilakukan berdasarkan bagian kode yang memang belum tercakup, bukan hanya menambah jumlah test secara sembarangan.

   b. **Menambahkan test case untuk setiap jalur logika**

      Jika payment.py mempunyai percabangan seperti:

      - If
      - Else if
      - Else

      Maka, test case perlu dibuat agar setiap kemungkinan jalur tersebut dapat dieksekusi.

      Sebagai contoh, apabila sistem pembayaran mempunyai kondisi:

      - pembayaran berhasil,
      - saldo tidak mencukupi,
      - jumlah pembayaran tidak valid,
      - metode pembayaran tidak tersedia,

      Maka setiap kondisi tersebut sebaiknya mempunyai test case tersendiri.

      Hal ini sesuai dengan konsep White-box Testing karena pengujian tidak hanya melihat output akhir, tetapi juga memastikan jalur logika program dieksekusi.

c. **Menambahkan pengujian terhadap nilai batas**

Selain kondisi normal, perlu ditambahkan test terhadap nilai batas atau *boundary condition*.

Contohnya apabila terdapat validasi jumlah pembayaran, test dapat diberikan pada kondisi seperti:

amount = 0
amount < 0
amount = batas minimum
amount = batas maksimum
amount > batas maksimum

Pengujian nilai batas penting karena kesalahan program sering terjadi di sekitar batas suatu kondisi.

d. Menguji kondisi error dan exception

Jika payment.py mempunyai kode yang melempar exception atau mengembalikan pesan kesalahan, maka bagian tersebut juga harus mempunyai unit test.

Contohnya:

input tidak valid
saldo tidak mencukupi
data pembayaran null
transaksi gagal

Dengan demikian, test tidak hanya mencakup *happy path* atau kondisi ketika transaksi berhasil, tetapi juga mencakup *negative path* dan kondisi error.

Materi Pertemuan 4 juga menjelaskan bahwa salah satu solusi ketika coverage rendah adalah menambahkan test case untuk path yang belum tercakup.

Jadi, untuk meningkatkan coverage payment.py, pengujian harus difokuskan pada 8 statement yang masih berstatus Miss, kemudian dibuat test case yang dapat mengeksekusi seluruh jalur, kondisi batas, dan kondisi error yang belum tercakup.

5. Menurut Anda, apakah coverage 78% sudah cukup baik? Jelaskan alasannya

Menurut saya, coverage sebesar 78% belum dapat dikatakan cukup baik jika dibandingkan dengan target coverage yang digunakan pada materi Pertemuan 4. Dalam materi dosen, target yang disarankan untuk statement coverage dan line coverage berada pada kisaran 80%–90%, sedangkan coverage project pada soal hanya sebesar 78%.

Artinya, nilai tersebut masih berada sedikit di bawah batas bawah target yang disarankan. Selain itu, pada konfigurasi JaCoCo di materi juga diberikan contoh threshold minimum sebesar 0,80 atau 80% untuk line coverage, sehingga coverage 78% masih belum memenuhi standar minimum tersebut. Meskipun demikian, angka 78% tetap menunjukkan bahwa sebagian besar source code sudah berhasil dieksekusi oleh test case, yaitu 39 dari 50 statement, sedangkan masih terdapat 11 statement yang belum dieksekusi. Permasalahan utamanya juga tidak hanya terletak pada angka total coverage, tetapi pada distribusi coverage antar file, karena terdapat file dengan coverage yang sudah tinggi seperti calculator.py sebesar 100%, namun masih ada file seperti payment.py yang hanya mencapai 60%. Kondisi ini menunjukkan bahwa masih terdapat bagian program tertentu yang belum diuji secara memadai dan perlu ditambahkan test case agar coverage menjadi lebih merata serta mendekati target yang telah ditetapkan.

Sebagai contoh:

```
calculator.py       = 100%
login_system.py     = 87%
payment.py          = 60%
utils.py            = 80%
```

Walaupun calculator.py sudah mencapai coverage 100%, file payment.py masih berada pada 60%. Hal ini menunjukkan bahwa terdapat bagian tertentu dari aplikasi yang belum memperoleh pengujian yang memadai.

Menurut saya, kondisi seperti ini masih perlu diperbaiki karena sebuah file dengan coverage rendah dapat menyimpan jalur logika atau kondisi error yang belum pernah diuji.

Namun, perlu dipahami juga bahwa code coverage yang tinggi tidak secara otomatis menjamin bahwa kualitas perangkat lunak sudah sempurna. Code coverage hanya menunjukkan seberapa banyak kode yang dieksekusi oleh test, bukan apakah seluruh test case mempunyai kualitas yang baik atau apakah seluruh kemungkinan kesalahan sudah ditemukan.

Oleh karena itu, target peningkatan coverage sebaiknya tidak hanya mengejar angka, tetapi juga memastikan test case mencakup:

- jalur logika penting,
- kondisi normal,
- kondisi error,
- nilai batas,
- exception,
- dan fungsi yang mempunyai risiko tinggi.

Bagian E: Studi Kasus Terpadu

Soal 8

1. **Implementasi Project**

   Pada studi kasus ini saya membuat project Java berbasis Maven dengan nama **`bank-account-ci`**. Project digunakan untuk mengimplementasikan class `BankAccount` yang memiliki tiga fungsi utama, yaitu `deposit(double amount)` untuk menambahkan saldo, `withdraw(double amount)` untuk mengurangi saldo, dan `getBalance()` untuk mengembalikan saldo saat ini.

   Pada proses implementasi juga diberikan validasi agar nilai transaksi tidak boleh negatif. Selain itu, proses withdraw tidak diperbolehkan apabila nominal penarikan lebih besar daripada saldo yang tersedia. Seluruh source code project disimpan pada repository GitHub sehingga kode program, unit test, konfigurasi Maven, dan workflow GitHub Actions dapat diperiksa langsung melalui repository.

2. **Tools dan Teknologi yang Digunakan**

   Project ini menggunakan beberapa tools dan teknologi, yaitu:

Tools/ Teknologi	Fungsi
Java 17	Bahasa dan environment utama project
Maven	Mengelola project, dependency, dan proses build
Junit 5	Framework untuk menjalankan unit testing
JaCoCo	Mengukur code coverage
Git	Version control untuk mencatat perubahan project
GitHub	Remote repository penyimpanan project
GitHub Actions	Menjalankan proses Continuous Integration secara otomatis

3. **Unit Testing**

   Sesuai ketentuan soal yang meminta minimal 6 test case, pada project ini saya membuat **9** unit test agar pengujian tidak hanya mencakup kondisi normal, tetapi juga nilai batas dan kondisi error.

   Test yang dilakukan meliputi:

   1) Memastikan saldo awal bernilai 0.

   2) Deposit dengan nominal valid.

3) Deposit lebih dari satu kali untuk memastikan saldo terakumulasi.

4) Deposit dengan nilai 0.

5) Deposit dengan nilai negatif dan memastikan exception muncul.

6) Withdraw dengan nominal valid.

7) Withdraw seluruh saldo.

8) Withdraw dengan nominal lebih besar dari saldo dan memastikan exception muncul.

9) Withdraw dengan nilai negatif dan memastikan exception muncul.

Unit test dijalankan menggunakan

```
mvn test
```

Hasil pengujian menunjukkan:

```
Tests run: 9
Failures: 0
Errors: 0
Skipped: 0
BUILD SUCCESS
```

Berdasarkan hasil tersebut, seluruh test case berhasil dijalankan tanpa failure maupun error.

4. Code Coverage

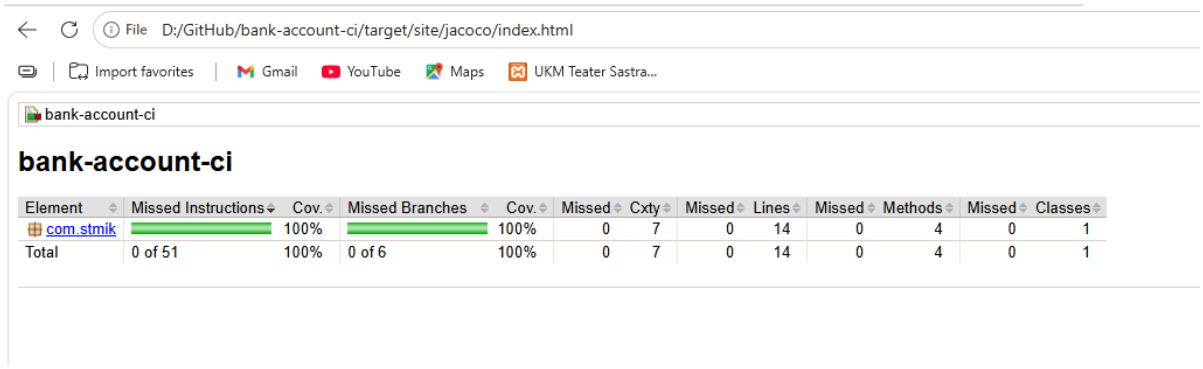
Setelah unit test berhasil dijalankan, code coverage diukur menggunakan **JaCoCo**. Coverage report dihasilkan melalui:

```
mvn jacoco:report
```

Hasil pengujian menunjukkan coverage sebagai berikut:

Jenis Coverage	Hasil
Instruction Coverage	100%
Branch Coverage	100%
Line Coverage	100%
Method Coverage	100%
Class Coverage	100%

JaCoCO menunjukkan bahwa tidak terdapat instruction, branch, line, method, maupun class yang terlewat oleh pengujian. Hal ini berarti seluruh bagian utama pada class BankAccount berhasil dieksekusi oleh unit test yang telah dibuat.



5. Implementasi GitHub Actions

Setelah unit test dan coverage berhasil dijalankan secara lokal, project kemudian diintegrasikan dengan GitHub Actions.

Workflow dibuat untuk menjalankan proses berikut secara otomatis:

1) Checkout code, untuk mengambil source code dari repository.
2) Setup JDK 17, untuk menyiapkan environment Java.
3) Run tests, untuk menjalankan unit test menggunakan Maven.
4) Generate coverage report, untuk menghasilkan laporan JaCoCo.
5) Upload coverage report, untuk mengunggah hasil coverage sebagai artifact.

Workflow akan dijalankan ketika terjadi push ke branch main atau develop, serta ketika terdapat pull_request ke branch main.

6. Hasil Pipeline GitHub Actions

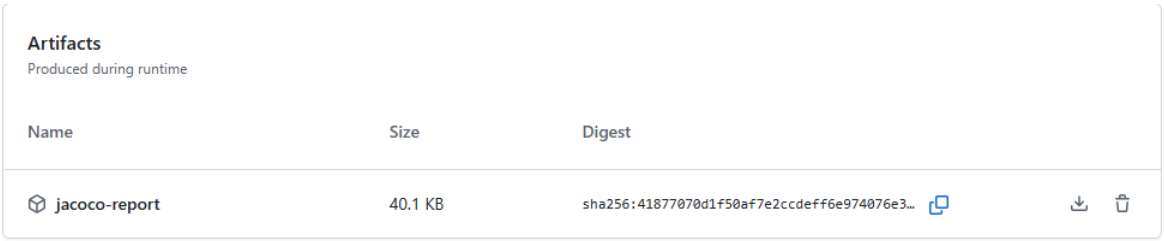
Setelah akan dijalankan ketika terjadi push ke branch main atau develop, serta ketika terdapat pull_request ke branch main.

Hasil pipeline menunjukkan job test berstatus **succeeded**, dan seluruh proses berhasil dijalankan:

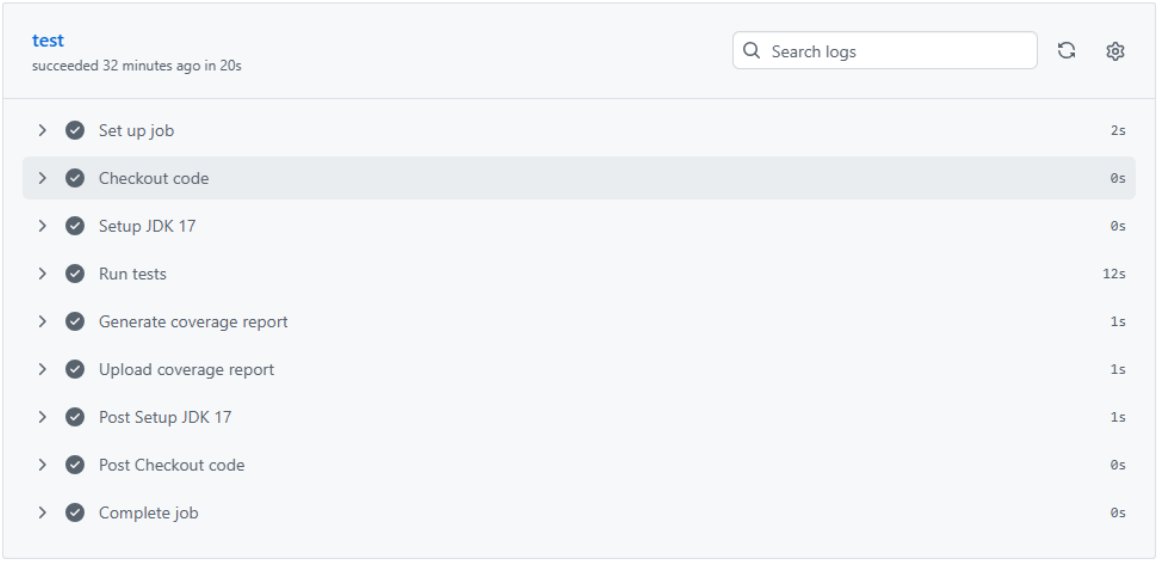
```
Checkout code                → Success
Setup JDK 17                 → Success
Run tests                    → Success
Generate coverage report     → Success
Upload coverage report       → Success
```

Selain itu, GitHub Actions berhasil menghasilkan artifact bernama:

```
jacoco-report
```



Hal ersebut menunjukkan bahwa proses Continuous Integration telah berjalan dengan baik, mulai dari pengujian unit hingga pembuatan serta penyimpanan laporan coverage.



7. Repository GitHub

Seluruh source code, unit test, konfigurasi Maven, dan workflow GitHub Actions dapat dilihat pada repository berikut:

Repository:

https://github.com/Selpiya/bank-account-ci

Struktur utama repository terdiri atas:

```
bank-account-ci/
├── .github/workflows/
├── src/main/java/com/stmik/
├── src/test/java/com/stmik/
├── .gitignore
└── pom.xml
```

Kesimpulan

Project BankAccount berhasil diimplementasikan sesuai ketentuan soal. Project menggunakan Java 17 dan Maven, dengan JUnit 5 sebagai framework unit testing dan JaCoCo untuk mengukur code coverage. Sebanyak 9 test case berhasil dijalankan tanpa failure maupun error, sehingga telah melebihi ketentuan minimal 6 test case.

Hasil JaCoCo menunjukkan 100% instruction, branch, line, method, dan class coverage. Project juga berhasil diintegrasikan dengan GitHub Actions, dan seluruh step pada pipeline berjalan dengan status succeeded serta menghasilkan artifact jacoco-report.

Seluruh kode program dan konfigurasi teknis tersedia pada repository GitHub, sedangkan pada laporan ini ditampilkan proses implementasi, hasil pengujian, hasil coverage, dan bukti keberhasilan pipeline.

Link Repository GitHub

Keperluan/ Soal	Repository	Link
Soal 2 – Praktik Git Commands	latihan-git-3224007	https://github.com/Selpiya/latihan-git-3224007.git
Soal 6 – Java Unit Test & Github Actions	student-grace-ic	https://github.com/Selpiya/student-grade-ci.git
Soal 8 – Studi Kasus Terpadu BankAccount	bank-account-ci	https://github.com/Selpiya/bank-account-ci.git